```python
import sqlite3
import grpc
from concurrent import futures
from contextlib import contextmanager
from datetime import datetime, timezone
from pathlib import Path
from uuid import uuid4

import banking_pb2
import banking_pb2_grpc


DB_PATH = Path(__file__).resolve().with_name("banking.db")


class BankData:
    def __init__(self, database_path=DB_PATH):
        self.database_path = str(database_path)
        self.initialize_database()

    @contextmanager
    def connect(self):
        # Open a separate connection for each request (safe with gRPC threads).
        connection = sqlite3.connect(self.database_path, timeout=10)
        try:
            connection.execute("PRAGMA foreign_keys = ON")
            yield connection
        finally:
            connection.close()

    def initialize_database(self):
        with self.connect() as conn, conn:
            conn.execute("""
                CREATE TABLE IF NOT EXISTS accounts (
                    user_id TEXT PRIMARY KEY,
                    balance_paise INTEGER NOT NULL CHECK (balance_paise >= 0)
                )
            """)
            conn.execute("""
                CREATE TABLE IF NOT EXISTS transactions (
                    transaction id TEXT PRIMARY KEY.
```

```python
import sqlite3
import grpc
from concurrent import futures
from contextlib import contextmanager
from datetime import datetime, timezone
from pathlib import Path
from uuid import uuid4

import banking_pb2
import banking_pb2_grpc


DB_PATH = Path(__file__).resolve().with_name("banking.db")


class BankData:
    def __init__(self, database_path=DB_PATH):
        self.database_path = str(database_path)
        self.initialize_database()

    @contextmanager
    def connect(self):
        # Open a separate connection for each request (safe with gRPC threads).
        connection = sqlite3.connect(self.database_path, timeout=10)
        try:
            connection.execute("PRAGMA foreign_keys = ON")
            yield connection
        finally:
            connection.close()

    def initialize_database(self):
        with self.connect() as conn, conn:
            conn.execute("""
                CREATE TABLE IF NOT EXISTS accounts (
                    user_id TEXT PRIMARY KEY,
                    balance_paise INTEGER NOT NULL CHECK (balance_paise >= 0)
                )
            """)
            conn.execute("""
                CREATE TABLE IF NOT EXISTS transactions (
```

```
            CREATE TABLE IF NOT EXISTS transactions (
                transaction_id TEXT PRIMARY KEY,
                from_user_id TEXT NOT NULL,
                to_user_id TEXT NOT NULL,
                amount_paise INTEGER NOT NULL CHECK (amount_paise > 0),
                timestamp TEXT NOT NULL,
                FOREIGN KEY (from_user_id) REFERENCES accounts(user_id),
                FOREIGN KEY (to_user_id) REFERENCES accounts(user_id)
            )
        """)
        # Seed the three demo accounts only if they do not already exist.
        conn.executemany(
            "INSERT OR IGNORE INTO accounts (user_id, balance_paise) VALUES (?, ?)",
            [
                ("101", 1000000),  # Alice: Rs. 10,000
                ("102", 500000),   # Bob: Rs. 5,000
                ("103", 800000),   # Charlie: Rs. 8,000
            ],
        )


class AccountManagementService(
    banking_pb2_grpc.AccountManagementServiceServicer
):
    def __init__(self, bank):
        self.bank = bank

    def getBalance(self, request, context):
        with self.bank.connect() as conn:
            row = conn.execute(
                "SELECT balance_paise FROM accounts WHERE user_id = ?",
                (request.user_id,),
            ).fetchone()

        if row is None:
            context.abort(grpc.StatusCode.NOT_FOUND, "Account does not exist.")

        return banking_pb2.BalanceResponse(
            user_id=request.user_id,
            balance_paise=row[0],
```

```
            balance_paise=row[0],
        )

    def updateBalance(self, request, context):
        if request.new_balance_paise < 0:
            context.abort(
                grpc.StatusCode.INVALID_ARGUMENT,
                "Account balance cannot be negative.",
            )

        with self.bank.connect() as conn, conn:
            cursor = conn.execute(
                "UPDATE accounts SET balance_paise = ? WHERE user_id = ?",
                (request.new_balance_paise, request.user_id),
            )
            if cursor.rowcount == 0:
                context.abort(grpc.StatusCode.NOT_FOUND, "Account does not exist.")

        return banking_pb2.UpdateBalanceResponse(
            success=True,
            message="Account balance updated successfully.",
            updated_balance_paise=request.new_balance_paise,
        )


class TransactionService(
    banking_pb2_grpc.TransactionServiceServicer
):
    def __init__(self, bank):
        self.bank = bank

    def initiateTransfer(self, request, context):
        sender = request.from_user_id
        receiver = request.to_user_id
        amount = request.amount_paise

        if amount <= 0:
            context.abort(
                grpc.StatusCode.INVALID_ARGUMENT,
                "Transfer amount must be greater than zero.",
```

```
            "Transfer amount must be greater than zero.",
        )
    if sender == receiver:
        context.abort(
            grpc.StatusCode.INVALID_ARGUMENT,
            "Cannot transfer money to the same account.",
        )

    transaction_id = str(uuid4())
    timestamp = datetime.now(timezone.utc).isoformat()

    # One atomic transaction: balances and transaction record commit together.
    with self.bank.connect() as conn, conn:
        conn.execute("BEGIN IMMEDIATE")
        sender_row = conn.execute(
            "SELECT balance_paise FROM accounts WHERE user_id = ?", (sender,)
        ).fetchone()
        if sender_row is None:
            context.abort(grpc.StatusCode.NOT_FOUND, "Sender account does not exist.")

        receiver_row = conn.execute(
            "SELECT balance_paise FROM accounts WHERE user_id = ?", (receiver,)
        ).fetchone()
        if receiver_row is None:
            context.abort(grpc.StatusCode.NOT_FOUND, "Receiver account does not exist.")

        if sender_row[0] < amount:
            context.abort(grpc.StatusCode.FAILED_PRECONDITION, "Insufficient funds.")

        conn.execute(
            "UPDATE accounts SET balance_paise = balance_paise - ? WHERE user_id = ?",
            (amount, sender),
        )
        conn.execute(
            "UPDATE accounts SET balance_paise = balance_paise + ? WHERE user_id = ?",
            (amount, receiver),
        )
        conn.execute(
            """INSERT INTO transactions
               (transaction_id, from_user_id, to_user_id, amount_paise, timestamp)
```

```
                (transaction_id, from_user_id, to_user_id, amount_paise, timestamp)
                VALUES (?, ?, ?, ?, ?)""",
            (transaction_id, sender, receiver, amount, timestamp),
        )

    return banking_pb2.TransferResponse(
        success=True,
        message="Money transferred successfully.",
        transaction_id=transaction_id,
    )

def getTransactionHistory(self, request, context):
    with self.bank.connect() as conn:
        if conn.execute(
            "SELECT 1 FROM accounts WHERE user_id = ?", (request.user_id,)
        ).fetchone() is None:
            context.abort(grpc.StatusCode.NOT_FOUND, "Account does not exist.")

        rows = conn.execute(
            """SELECT transaction_id, from_user_id, to_user_id,
                      amount_paise, timestamp
               FROM transactions
               WHERE from_user_id = ? OR to_user_id = ?
               ORDER BY rowid""",
            (request.user_id, request.user_id),
        ).fetchall()

    history = [
        banking_pb2.Transaction(
            transaction_id=row[0],
            from_user_id=row[1],
            to_user_id=row[2],
            amount_paise=row[3],
            timestamp=row[4],
        )
        for row in rows
    ]
    return banking_pb2.TransactionHistoryResponse(transactions=history)
```

```python
def serve():
    bank = BankData()
    server = grpc.server(futures.ThreadPoolExecutor(max_workers=10))

    banking_pb2_grpc.add_AccountManagementServiceServicer_to_server(
        AccountManagementService(bank), server
    )
    banking_pb2_grpc.add_TransactionServiceServicer_to_server(
        TransactionService(bank), server
    )

    address = "127.0.0.1:50051"
    bound_port = server.add_insecure_port(address)
    if bound_port == 0:
        raise RuntimeError(f"Failed to bind server to {address}")

    server.start()
    print("Banking gRPC Server started successfully!")
    print(f"SQLite database: {bank.database_path}")
    print(f"Listening on {address}")
    print("Press Ctrl+C to stop the server.")

    try:
        server.wait_for_termination()
    except KeyboardInterrupt:
        server.stop(grace=5).wait()
        print("\nBanking gRPC Server stopped.")


if __name__ == "__main__":
    serve()
```